

# Detección de Anomalías Utilizando Autocodificadores

Autor: Matías Cisnero

Tutora: Dra. M. Juliana Gambini

Diciembre 2025

Universidad Nacional de Hurlingham





# Índice General

|                                                    |           |
|----------------------------------------------------|-----------|
| <b>1. Introducción</b>                             | <b>11</b> |
| 1.1. Motivación . . . . .                          | 11        |
| 1.2. Estado del Arte . . . . .                     | 11        |
| <b>2. Materiales y Métodos</b>                     | <b>13</b> |
| 2.1. Conjunto de datos y AED . . . . .             | 13        |
| 2.2. Descripción de los métodos . . . . .          | 18        |
| 2.2.1. Autocodificadores . . . . .                 | 18        |
| 2.2.2. Métricas de evaluación del modelo . . . . . | 23        |
| <b>3. Resultados</b>                               | <b>27</b> |
| <b>4. Conclusiones</b>                             | <b>33</b> |



# Índice de Figuras

|      |                                                                                                                                 |    |
|------|---------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1. | Ejemplo de un autocodificador, donde se codifica y decodifica una imagen (tomado de [1]).                                       | 13 |
| 2.2. | Boxplot combinado de <i>Area Mean</i> , <i>Fractal Dimension Mean</i> y <i>Simmetry Mean</i> , segmentado por (normal/anómalo). | 15 |
| 2.3. | Matriz de correlación entre los atributos del conjunto de datos.                                                                | 16 |
| 2.4. | Esquema de la división del conjunto de datos.                                                                                   | 18 |
| 2.5. | Arquitectura general de un autocodificador.                                                                                     | 19 |
| 2.6. | Funciones de activación ReLU y GELU.                                                                                            | 20 |
| 3.1. | Distribución del error de reconstrucción para el autocodificador A.                                                             | 28 |
| 3.2. | Distribución del error de reconstrucción para el autocodificador B.                                                             | 29 |
| 3.3. | Distribución del error de reconstrucción para el autocodificador C.                                                             | 29 |
| 3.4. | Curvas ROC y AUC obtenidos para los modelos AE, SAE y CAE entrenados con el Conjunto A (0 % de anomalías).                      | 30 |
| 3.5. | Curvas ROC y AUC obtenidos para los modelos AE, SAE y CAE entrenados con el Conjunto B (10 % de anomalías).                     | 31 |
| 3.6. | Curvas ROC y AUC obtenidos para los modelos AE, SAE y CAE entrenados con el Conjunto C (25 % de anomalías).                     | 31 |



# Índice de Tablas

|      |                                                                                                                                                    |    |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1. | Descripción de las diez características base extraídas de las imágenes digitalizadas de FNA.                                                       | 14 |
| 2.2. | Distribución de la variable objetivo <i>diagnosis</i>                                                                                              | 15 |
| 2.3. | División del conjunto original en entrenamiento, validación y prueba, manteniendo la proporción original de clases.                                | 17 |
| 2.4. | Subconjuntos derivados del subconjunto de entrenamiento para evaluar la resistencia del autocodificador ante diferentes proporciones de anomalías. | 18 |
| 2.5. | Matriz de confusión. La diagonal verde representa predicciones correctas, mientras que las celdas rojas corresponden a errores de clasificación.   | 24 |
| 3.1. | Hiperparámetros de entrenamiento para cada modelo.                                                                                                 | 27 |
| 3.2. | Valores de umbral $\epsilon$ obtenidos con diferentes métodos y modelos.                                                                           | 28 |
| 3.3. | Métricas de rendimiento para cada modelo entrenado con los conjuntos A, B y C.                                                                     | 30 |

## Glosario

| Notación                                   | Descripción                                                                                   |
|--------------------------------------------|-----------------------------------------------------------------------------------------------|
| $\mathbf{X} = \{x^{(1)}, \dots, x^{(N)}\}$ | Conjunto de datos (N ejemplos)                                                                |
| $X_i$                                      | Atributo $i$ -ésimo del conjunto de datos                                                     |
| $x^{(u)} \in \mathbb{R}^D$                 | Ejemplo $u$ -ésimo del conjunto de datos                                                      |
| $x_k^{(u)}$                                | Componente $k$ del ejemplo $x^{(u)}$                                                          |
| $\hat{x}^{(u)}$                            | Reconstrucción del ejemplo $x^{(u)}$                                                          |
| $z^{(u)} \in \mathbb{R}^K$                 | Representación latente del ejemplo $u$                                                        |
| $z_j^{(u)}$                                | Activación del componente $j$ de la representación latente                                    |
| $D$                                        | Dimensión del vector de entrada                                                               |
| $K$                                        | Dimensión de la capa latente                                                                  |
| $H$                                        | Dimensión de una capa oculta genérica                                                         |
| $N$                                        | Cantidad total de ejemplos                                                                    |
| $W$                                        | Conjunto de matrices de pesos                                                                 |
| $w_{jk}$                                   | Peso que conecta la entrada $k$ con la neurona $j$                                            |
| $b$                                        | Conjunto de vectores sesgo o <i>bias</i>                                                      |
| $b_j$                                      | Sesgo o <i>bias</i> de la neurona $j$                                                         |
| $a_j^{(u)}$                                | Activación de la neurona $j$ para el ejemplo $u$                                              |
| $g(x)$                                     | Función de activación                                                                         |
| $\mathcal{L}(W, b)$                        | Función de costo (MSE)                                                                        |
| $\bar{x}_i$                                | Media muestral del atributo $X_i$                                                             |
| $s_{x_i}$                                  | Desvío estándar muestral del atributo $X_i$                                                   |
| $\text{Cov}(X_i, X_j)$                     | Covarianza entre atributos $X_i$ y $X_j$                                                      |
| $\text{Corr}(X_i, X_j)$                    | Correlación entre atributos $X_i$ y $X_j$                                                     |
| $\Sigma \in \mathbb{R}^{D \times D}$       | Matriz de correlaciones                                                                       |
| $\rho$                                     | Activación promedio deseada (SAE)                                                             |
| $\hat{\rho}_j$                             | Activación promedio real de la neurona $j$ (SAE)                                              |
| $\text{KL}(p \parallel q)$                 | Divergencia Kullback–Leibler                                                                  |
| $\mathcal{L}_{\text{sparse}}(W, b)$        | Función de costo por dispersidad (SAE)                                                        |
| $J_f(x^{(u)})$                             | Jacobiano del codificador evaluado en $x^{(u)}$ (CAE)                                         |
| $\ A\ _F^2$                                | Cuadrado de la norma de Frobenius de la matriz $A$                                            |
| $\mathcal{L}_{\text{CAE}}(W, b)$           | Función de costo total del CAE (reconstrucción + penalización contractiva)                    |
| $\mathbf{T}$                               | Conjunto de datos con igual distribución que $\mathbf{X}$                                     |
| $t$                                        | Dato de entrada con igual distribución que los datos pertenecientes a la clase “normal”       |
| $\hat{t}$                                  | Reconstrucción de $t$ generada por el autocodificador                                         |
| $t^*$                                      | Dato atípico con distinta distribución que $t$                                                |
| $\hat{t}^*$                                | Reconstrucción de $t^*$ generada por el autocodificador                                       |
| $\epsilon$                                 | Umbral calculado a partir del error de reconstrucción de los datos considerados normales      |
| $y^{(u)}$                                  | Etiqueta del ejemplo $x^{(u)}$ , donde 0 indica normal (benigno) y 1 indica anómalo (maligno) |
| $\hat{y}^{(u)}$                            | Predicción de la clase de $x^{(u)}$ por el autocodificador según el umbral $\epsilon$         |



# Resumen

El presente informe aborda la detección de anomalías en datos biomédicos utilizando el conjunto *Breast Cancer Wisconsin (Diagnostic)*, cuyo objetivo es distinguir patrones malignos a partir de muestras benignas obtenidas mediante punción con aguja fina (FNA). Tras un análisis exploratorio inicial, incluyendo distribuciones mediante *boxplots* y la matriz de correlación. Se implementan autocodificadores entrenados con distintas proporciones de datos de pacientes sanos y enfermos, con el fin de identificar anomalías mediante el error de reconstrucción. Se evalúan tres variantes: el autocodificador clásico, el *sparse autoencoder* y el *contractive autoencoder*, todas desarrolladas en el entorno de *PyTorch*.

Los resultados, visualizados a través de tablas, figuras e histogramas muestran que, tal como dicta en la teoría, los datos anómalos presentan en la mayoría de casos un mayor error de reconstrucción y los modelos entrenados para tal clasificación muestran una buena capacidad discriminativa. Destacando al modelo *sparse autoencoder* por sus métricas generales y al *contractive autoencoder* por su mínimo valor de falsos positivos. El umbral utilizado para clasificar estos datos como normales o anomalías según su error de reconstrucción, se calcula utilizando el índice de Youden.



# Capítulo 1

## Introducción

La detección de anomalías es un campo fundamental dentro del análisis de datos, cuyo objetivo principal es la identificación de patrones que no se ajustan a un comportamiento que coincide con el de la mayoría de los registros. En el contexto de los datos de salud, una anomalía puede representar un error de medición, o una indicación temprana de una condición médica patológica, como el cáncer de mama.

Este trabajo se centra en desarrollar un sistema robusto para la detección automática de tales desviaciones. Se busca resolver la limitación de los métodos estadísticos tradicionales, los cuales a menudo no logran capturar las complejas relaciones no lineales presentes en conjuntos de datos de alta dimensionalidad. El problema específico que se aborda es cómo entrenar un **autocodificador** para que aprenda como identificar si la muestra tomada del área mamaria de un paciente, mediante la aspiración con aguja fina (FNA) se trata de un caso benigno o maligno a partir de un conjunto de datos.

### 1.1. Motivación

El diagnóstico temprano de enfermedades como el cáncer de mama es una tarea prioritaria en la medicina moderna. En muchos casos, los datos obtenidos a través de las muestras contienen patrones sutiles que pueden pasar desapercibidos a través de métodos tradicionales de análisis. Por este motivo, resulta fundamental el desarrollo de herramientas automáticas capaces de detectar irregularidades de manera precisa y rápida.

Las redes neuronales, y en particular los autocodificadores, ofrecen un enfoque adecuado para esta tarea. Un autocodificador es un tipo de red neuronal diseñado para aprender una representación comprimida de los datos, capturando sus características más relevantes. Esta capacidad lo convierte en un buen candidato para identificar si un nuevo registro sigue la misma distribución que los registros normales con los que fue entrenado el modelo.

La incorporación de este tipo de modelos en sistemas de monitoreo podría asistir a profesionales de la salud en la detección temprana de pacientes en riesgo. Por ello, en este proyecto se emplea un autocodificador como herramienta para analizar muestras tomadas de pacientes y explorar su utilidad como detector de posibles casos de cáncer de mama.

### 1.2. Estado del Arte

En esta sección se revisan los principales avances en métodos basados en autocodificadores para la detección de anomalías en distintos contextos como imágenes, datos biomédicos y tráfico de red, entre otros.

Al hablar de autocodificadores se hace referencia a un tipo de red neuronal, la cual tiene como propósito comprimir y reconstruir los datos de entrada (datos tabulares, imágenes, etc). Este tipo de modelo surge de las primeras investigaciones en redes neuronales orientadas a aprender representaciones internas representativas sin una supervisión explícita. De los aportes más trascendentes fue el realizado por los autores Rumelhart, Hinton y Williams [5], quienes introdujeron el uso del algoritmo de retropropagación para aprender representaciones internas compactas y características.

Posteriormente, el enfoque fue formalizado por Bourlard y Kamp [3]. Los mismos demostraron la relación de este tipo de red neuronal, con métodos clásicos de reducción de dimensionalidad, como la descomposición en valores singulares. Estos desarrollos sentaron las bases del autocodificador como un modelo para el aprendizaje de representaciones de baja dimensionalidad.

En la actualidad existen múltiples variantes de los autocodificadores, como los evaluados por los autores Nelay y Turgeon [8]. Los mismos plantean 11 arquitecturas variantes de autocodificadores y evalúan sus capacidades de reconstrucción, generación de muestras, visualización del espacio latente y precisión en la clasificación de datos anómalos. Algunas de las arquitecturas utilizadas son *Denosing Autoencoder*, *Sparse Autoencoder*, *Contractive Autoencoder* y *Variational Autoencoder* junto a algunas de sus variantes (VQ-VAE,  $\beta$ -VAE, etc). Para llevar a cabo las comparaciones, utilizan los conjuntos de datos Fashion-MNIST (FMNIST) y MNIST, donde clasifican las anomalías según el error de reconstrucción, el cual es mucho mayor en datos anómalos. Los resultados obtenidos demuestran una mayor capacidad de detección de anomalías en los modelos IWAE,  $\beta$ -VAE y adVAE.

Otra variante fue la propuesta por Walczyna et al. [12]. Los cuales comparan tres arquitecturas: autocodificadores, *Variational Autoencoder* y una variante de su autoría de *Variational Autoencoder* que utiliza otra función de costo denominada como M-ELBO. Esta permite entrenar al *Variational Autoencoder* con todos los datos (no solo los normales), pero solo aprendiendo a reconstruir los datos no anómalos. Para este cometido, utilizaron el conjunto de datos InSDN y concluyeron en que la mejor arquitectura para la detección de anomalías es su versión modificada del *Variational Autoencoder*. Esto muestra la importancia de incluir datos anómalos en el entrenamiento a pesar de no reconstruirlos directamente.

Por otra parte, los autores Bouman y Heskes [2], cuestionan la capacidad de los autocodificadores para detectar anomalías mediante el error de reconstrucción. Los mismos comprueban matemáticamente que las anomalías lejanas a los datos normales pueden ser reconstruidas perfectamente, denotando una falta de fiabilidad en los modelos. Para eso, realizan pruebas en el conjunto de datos MNIST, donde observan que el codificador del modelo presenta áreas en donde se reduce la codificación a una simple transformación lineal. Concluyen que este problema predomina con activaciones lineales como ReLU, aunque con las sigmoides también ocurre, pero en menor medida.

Finalmente, los autores Nawaz et al. [7] realizan una revisión de los ensambles de autocodificadores o EoAEs (por sus siglas en inglés de *Ensemble of Autoencoders*) en múltiples conjuntos de datos biomédicos, como alternativa más robusta al uso de un solo modelo de clasificación. En los EoAEs se utilizan distintas maneras para clasificar las anomalías teniendo en cuenta el error de reconstrucción de cada autocodificador que lo compone. Entre estas maneras se encuentra el método de la media, de la votación y de la mediana, entre otros. En los casos revisados se demuestra que los EoAEs pueden funcionar mejor que un solo autocodificador para diversas aplicaciones biomédicas de detección de anomalías (ECG, EEG, MRI, X-RAY, etc).

## Capítulo 2

# Materiales y Métodos

En este capítulo se describe el conjunto de datos utilizado, de dónde se obtuvo y un análisis exploratorio al mismo, con el propósito de observar las diversas relaciones y distribuciones de las variables.

Por otra parte, el método empleado para la detección de anomalías es el **Autocodificador (Auto-encoder)**. Un autocodificador es un tipo de red neuronal artificial diseñada para aprender una representación codificada de un conjunto de datos. La arquitectura se compone de dos partes principales: el **Codificador (Encoder)** y el **Decodificador (Decoder)**. El funcionamiento general de esta arquitectura puede observarse en la Figura 2.1.

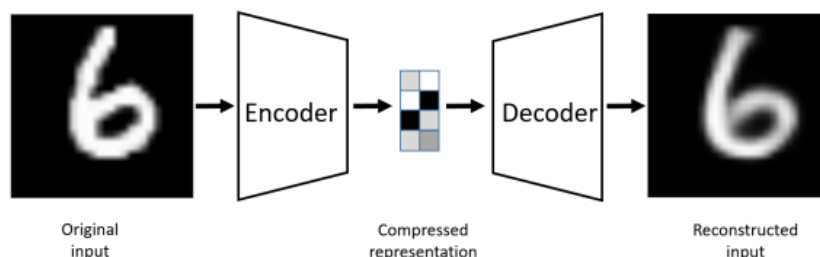


Figura 2.1: Ejemplo de un autocodificador, donde se codifica y decodifica una imagen (tomado de [1]).

### 2.1. Conjunto de datos y AED

Para la realización de este trabajo, se utiliza el conjunto de datos *Breast Cancer Wisconsin (Diagnostic) Data Set*, el cual se obtuvo de la plataforma Kaggle [6], que es una copia del dataset original publicado por Wolberg et al. [13]. Este conjunto de datos es ampliamente utilizado en tareas de clasificación y análisis de anomalías en el ámbito biomédico. Antes de aplicar cualquier técnica de modelado, se realiza un análisis exploratorio de datos (AED) con el objetivo de comprender la distribución de las características, detectar posibles valores atípicos y observar la relación entre las variables de las muestras obtenidas.

El conjunto de datos está compuesto por 569 registros de muestras citológicas y 32 atributos, de los cuales 30 corresponden a características numéricas derivadas del análisis de imágenes digitalizadas, correspondientes a aspiraciones con aguja fina de masas mamarias. Las 2 características restantes son categóricas, *ID*, el cual es solo un metadato que no se utiliza para el entrenamiento del modelo y el **diagnóstico** que puede ser benigno o maligno. Las muestras incluidas en el conjunto de datos fueron recolectadas entre enero de 1989 y noviembre de 1991.

Las **muestras citológicas** se refieren a grupos celulares obtenidas de las masas mamarias mediante la técnica de aspiración con aguja fina (FNA), que es un procedimiento poco invasivo utilizado para obtener material celular mediante la punción de la masa mamaria con una aguja de pequeño calibre. Esta técnica, ampliamente usada en el diagnóstico de tumores mamarios, permite recolectar células para su posterior análisis citológico, evitando métodos quirúrgicos más invasivos. Su uso en el estudio del cáncer de mama se remonta a la década de 1930 y ha demostrado poseer una elevada precisión diagnóstica cuando es ejecutada e interpretada por un patólogo especializado.

La FNA posibilita examinar características celulares y arquitectónicas del tejido aspirado, así como obtener material apto para tinciones citológicas que contribuyen a la detección de alteraciones malignas. Debido a su eficacia, simplicidad y bajo riesgo, la técnica se ha consolidado como una herramienta fundamental en la evaluación citológica de lesiones mamarias [4].

Las diez características base del conjunto de datos fueron obtenidas mediante las metodologías computacionales detalladas en el artículo original de Street et al. [11]. Estas características describen propiedades estructurales de los núcleos celulares, tales como tamaño, forma, textura y simetría, y se muestran en la Tabla 2.1.

Tabla 2.1: Descripción de las diez características base extraídas de las imágenes digitalizadas de FNA.

| No. | Característica                                        | Descripción                                                    |
|-----|-------------------------------------------------------|----------------------------------------------------------------|
| 1   | <b>Radio</b> ( <i>radius</i> )                        | Media de las distancias del centro a los puntos del perímetro. |
| 2   | <b>Textura</b> ( <i>texture</i> )                     | Desviación estándar de los valores de la escala de grises.     |
| 3   | <b>Perímetro</b> ( <i>perimeter</i> )                 | Medida del contorno del núcleo.                                |
| 4   | <b>Área</b> ( <i>area</i> )                           | Superficie cubierta por el núcleo.                             |
| 5   | <b>Suavidad</b> ( <i>smoothness</i> )                 | Variación local en las longitudes del radio.                   |
| 6   | <b>Compacidad</b> ( <i>compactness</i> )              | $\text{Perímetro}^2 / \text{área} - 1,0$ .                     |
| 7   | <b>Concavidad</b> ( <i>concavity</i> )                | Severidad de las porciones cóncavas del contorno.              |
| 8   | <b>Puntos Cóncavos</b> ( <i>concave points</i> )      | Número de porciones cóncavas del contorno.                     |
| 9   | <b>Simetría</b> ( <i>symmetry</i> )                   | Nivel de simetría del núcleo celular.                          |
| 10  | <b>Dimensión Fractal</b> ( <i>fractal dimension</i> ) | Aproximación a la “línea de costa” menos 1.                    |

Para cada una de estas diez características se calculan tres medidas distintas, lo que completa los 30 atributos finales del conjunto de datos:

1. La **media** de la característica.
2. El **error estándar** (*Standard Error*, SE) de la característica.
3. El **'peor'** valor o más grande, el cual se calcula como la media de los tres valores más grandes de la característica.

Iniciando el proceso de **análisis exploratorio de datos (AED)**, se verifica la integridad del conjunto. Tras la revisión de todos los registros y atributos, se confirma la ausencia total de datos faltantes, lo que hizo innecesaria cualquier etapa de imputación o limpieza de datos previa.

Tras el análisis general de los atributos, se examina la **distribución de la variable objetivo**, *diagnosis*, con el fin de comprender el balance entre las clases presentes en el conjunto de datos. La distribución se muestra en la Tabla 2.2.

Tabla 2.2: Distribución de la variable objetivo *diagnosis*

| Valor       | Porcentaje (%) | Cantidad de registros |
|-------------|----------------|-----------------------|
| B (Benigno) | 62.74          | 357                   |
| M (Maligno) | 37.26          | 212                   |

Esta distribución revela un leve desbalance entre ambas categorías, algo esperable para este trabajo, donde se espera detectar una anomalía, es decir, un diagnóstico maligno.

Para examinar la **distribución de las características** en función de la naturaleza del registro (normal o anómalo), se genera un gráfico de caja múltiple (*boxplot*) para las variables seleccionadas. Esta visualización, mostrada en la Figura 2.2, es crucial para identificar diferencias en la dispersión y tendencia central, así como la presencia de **valores atípicos** en cada grupo. Al comparar las distribuciones, se observa que el *Area Mean* (área promedio) presenta la distinción más clara entre los datos normales y anómalos. En contraste, el *Fractal Dimension Mean* (dimensión fractal promedio) muestra una superposición significativa entre las categorías, y las diferencias son más sutiles para el *Symmetry Mean* (simetría promedio).

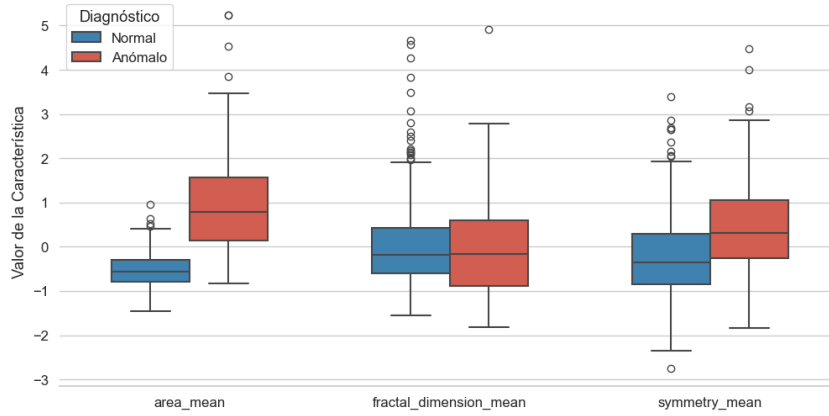


Figura 2.2: Boxplot combinado de *Area Mean*, *Fractal Dimension Mean* y *Symmetry Mean*, segmentado por (normal/anómalo).

Una vez analizada la distribución de las variables individualmente, se procede a calcular la matriz de correlación  $\Sigma$ , la cual mide el grado de relación entre cada par de atributos del conjunto de datos. Este valor está acotado entre -1 y 1, donde 1 indica una relación lineal perfecta, -1 una relación lineal inversa perfecta y 0 denota que no existe una relación lineal entre los atributos. La matriz de correlación se define en la Ecuación 2.1 para cada par de atributos  $(X_i, X_j)$  del conjunto de datos  $\mathbf{X}$ , el cual contiene  $N$  ejemplos y  $D$  atributos,  $\mathbf{X} \in \mathbb{R}^{N \times D}$ .

$$\Sigma = \begin{pmatrix} \text{Corr}(X_1, X_1) & \text{Corr}(X_1, X_2) & \cdots & \text{Corr}(X_1, X_d) \\ \text{Corr}(X_2, X_1) & \text{Corr}(X_2, X_2) & \cdots & \text{Corr}(X_2, X_d) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Corr}(X_d, X_1) & \text{Corr}(X_d, X_2) & \cdots & \text{Corr}(X_d, X_d) \end{pmatrix} \in \mathbb{R}^{d \times d}. \quad (2.1)$$

Donde la correlación entre dos atributos  $\text{Corr}(X_1, X_2)$  se muestra en la Ecuación 2.2.

$$\text{Corr}(X_i, X_j) = \frac{\text{Cov}(X_i, X_j)}{s_{x_i} s_{x_j}}, \quad (2.2)$$

y  $\text{Cov}(X_i, X_j)$  representa la covarianza entre  $X_i$  y  $X_j$ , mientras que  $s_{x_i}$  y  $s_{x_j}$  son sus desviaciones estándar. Finalmente, la covarianza utilizada en ese cálculo se expresa en la Ecuación 2.3, que mide la existencia de asociación lineal entre los dos atributos.

$$\text{Cov}(X_i, X_j) = \frac{1}{n} \sum_{t=1}^n (x_{ti} - \bar{x}_i)(x_{tj} - \bar{x}_j), \quad (2.3)$$

donde  $\bar{x}_i$  es la media muestral del atributo  $X_i$ , definida como:

$$\bar{x}_i = \frac{1}{n} \sum_{t=1}^n x_{ti}, \quad (2.4)$$

y  $s_{x_i}$  es el desvío estándar muestral asociado a  $X_i$ , dado por:

$$s_{x_i} = \sqrt{\frac{1}{n-1} \sum_{t=1}^n (x_{ti} - \bar{x}_i)^2}. \quad (2.5)$$

La matriz de correlación correspondiente a las características del conjunto de datos de este trabajo se presenta en la Figura 2.3, donde pueden observarse las relaciones lineales entre los distintos atributos. En la misma se observa una fuerte relación entre el área, el perímetro y el radio, algo esperable teniendo en cuenta las relaciones entre estas medidas, también se visualiza una leve relación inversa entre estas medidas y la media de la dimensión fractal.

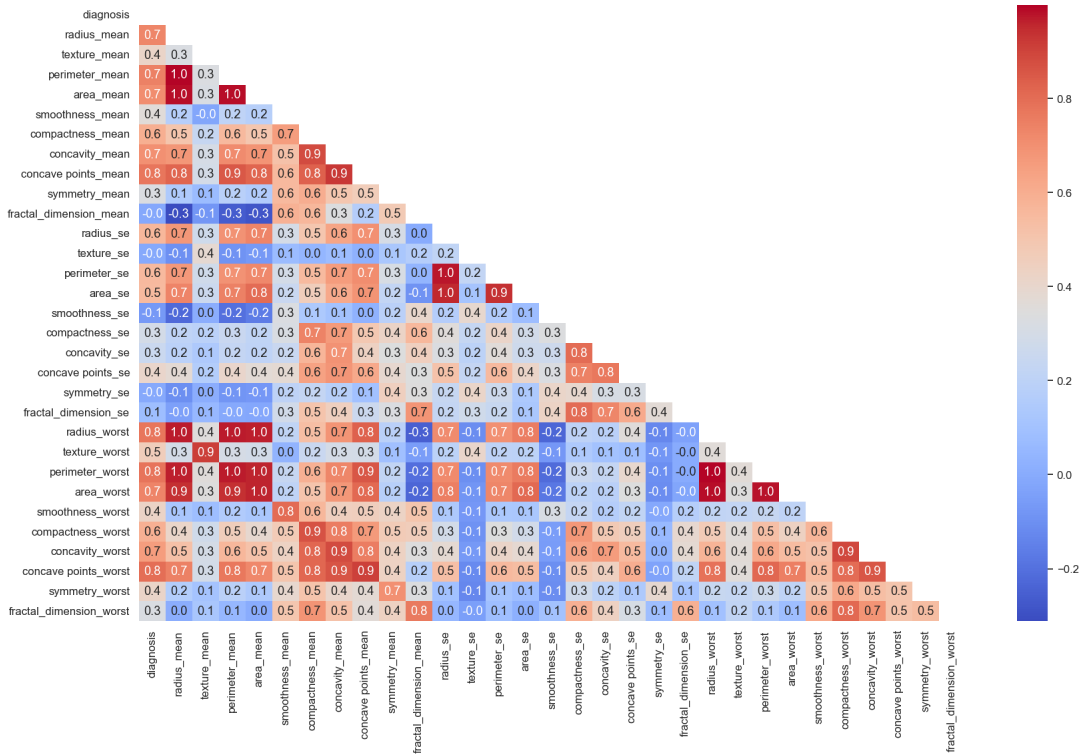


Figura 2.3: Matriz de correlación entre los atributos del conjunto de datos.

Para analizar la robustez del autocodificador frente a distintos niveles de contaminación en los datos, se realiza una división inicial del conjunto de datos en tres subconjuntos: **entrenamiento**, **validación** y **prueba**, manteniendo en todos la proporción original de clases, tal como se indica en el paso (1) de la Figura 2.4. En esta división, el 60 % de las muestras se destinó al entrenamiento, el 20 % al conjunto de validación y el 20 % restante al conjunto de prueba, tal como se muestra en la Tabla 2.3.



Tabla 2.3: División del conjunto original en entrenamiento, validación y prueba, manteniendo la proporción original de clases.

| Subconjunto   | Proporción | Total | Normales | Anómalos |
|---------------|------------|-------|----------|----------|
| Entrenamiento | 60 %       | 341   | 214      | 128      |
| Validación    | 20 %       | 114   | 72       | 42       |
| Prueba        | 20 %       | 114   | 72       | 42       |

El subconjunto de validación se utiliza con dos propósitos principales: evitar el sobreajuste (*overfitting*) del modelo durante el entrenamiento empleando únicamente los registros **normales** para este control y calcular las métricas necesarias durante el proceso de *grid search* para la selección de hiperparámetros.

Luego, se aplica una estandarización a todos los atributos de cada subconjunto de manera independiente. Este método transforma las variables para que tengan una media ( $\mu$ ) de cero y una desviación estándar ( $\sigma$ ) de uno, asegurando que todas las entradas del modelo tengan una escala comparable. Esta estandarización también contribuye a mitigar posibles problemas de *overflow* en las activaciones del autocodificador, los cuales pueden surgir cuando el modelo recibe valores excesivamente grandes. Este proceso se define formalmente en la Ecuación 2.6.

$$z = \frac{x - \mu}{\sigma}, \quad (2.6)$$

donde  $x$  corresponde al dato original y tanto  $\mu$  como  $\sigma$ , a la media y la desviación estándar de esa variable, respectivamente.

La estandarización de los tres subconjuntos (entrenamiento, validación y prueba) se realiza utilizando exclusivamente la media y el desvío estándar calculados a partir del **subconjunto de entrenamiento**. Esta estrategia evita la fuga de información o *data leakage*, fenómeno que ocurre cuando se incorporan, directa o indirectamente, estadísticas derivadas de los datos de validación o prueba durante el entrenamiento del modelo. La fuga de información genera una visión artificialmente optimista del desempeño, ya que introduce en el proceso señales que el modelo no debería conocer antes de ser evaluado.

A partir del subconjunto de entrenamiento, se generan tres nuevos subconjuntos con diferentes proporciones de anomalías, proceso señalado como paso (2) en la Figura 2.4. El objetivo es evaluar cómo afecta al modelo entrenar con distintos niveles de contaminación, manteniendo constante el tamaño total de cada conjunto y preservando la separación previa entre entrenamiento, validación y prueba establecida en el esquema general mostrado en la figura.

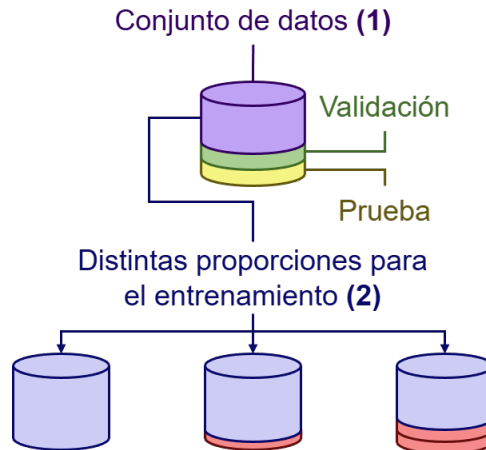


Figura 2.4: Esquema de la división del conjunto de datos.

La Tabla 2.4 resume las proporciones definidas para cada subconjunto derivado del entrenamiento. El subconjunto A contiene únicamente registros normales, mientras que B y C incorporan niveles crecientes de anomalías (10 % y 25 %, respectivamente).

Tabla 2.4: Subconjuntos derivados del subconjunto de entrenamiento para evaluar la resistencia del autocodificador ante diferentes proporciones de anomalías.

| ID | Proporción Anómalos | Proporción Normales | Registros Anómalos | Registros Normales |
|----|---------------------|---------------------|--------------------|--------------------|
| A  | 0 %                 | 100 %               | 0                  | 214                |
| B  | 10 %                | 90 %                | 21                 | 193                |
| C  | 25 %                | 75 %                | 53                 | 161                |

## 2.2. Descripción de los métodos

En esta sección se presentan los conceptos centrales utilizados en el desarrollo del trabajo. Primero, se describe formalmente la arquitectura del **autocodificador** clásico y dos de sus variantes: *sparse auto-encoder* y *contractive autoencoder*. Luego, el modo en que pueden emplearse como detector de anomalías, mediante el análisis del error de reconstrucción y estableciendo un criterio de decisión.

Con el criterio de decisión definido, se detallan las **métricas de evaluación** empleadas para cuantificar el desempeño del modelo, incluyendo las medidas estándar derivadas de la matriz de confusión, así como herramientas adicionales de análisis como la **curva ROC** y el **AUC**, que permiten evaluar el comportamiento del detector bajo diferentes umbrales de decisión.

### 2.2.1. Autocodificadores

El método empleado en este trabajo es el **autocodificador**, un tipo de perceptrón multicapa caracterizado por su estructura simétrica respecto a la capa latente y por poseer la misma dimensión tanto en la entrada como en la salida. Su objetivo es aprender una representación comprimida que permita reconstruir los datos originales.

La arquitectura de un autocodificador se compone de dos partes principales: un **codificador**, que transforma los datos de entrada en una representación de menor dimensionalidad, y un **decodificador**, que intenta reconstruir la entrada original a partir de dicha representación compacta. Entre ambos

se encuentra la **capa latente**, donde se almacena la representación interna de menor dimensión. La Figura 2.5 ilustra la arquitectura general empleada en este trabajo, mostrando el flujo desde la entrada  $X$ , su codificación en el espacio latente  $Z$ , y la posterior reconstrucción  $\hat{X}$  realizada por el decodificador.

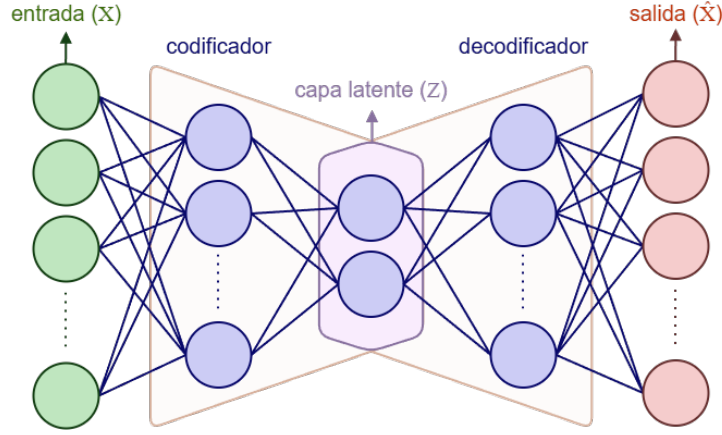


Figura 2.5: Arquitectura general de un autocodificador.

Para formalizar este proceso, considérese un conjunto de entrenamiento:

$$\mathbf{X} = \{x^{(1)}, x^{(2)}, \dots, x^{(N)}\}, \quad x^{(u)} \in \mathbb{R}^D, \quad u \in 1, \dots, N, \quad (2.7)$$

donde cada ejemplo puede escribirse como:

$$x^{(u)} = (x_1^{(u)}, x_2^{(u)}, \dots, x_D^{(u)}), \quad u \in 1, \dots, N, \quad (2.8)$$

y el conjunto de salidas deseadas se define como:

$$\hat{x}^{(u)} = x^{(u)}, \quad (2.9)$$

en la cual  $\hat{x}^{(u)}$  es la salida generada por el modelo y la salida esperada es la propia entrada  $x^{(u)}$ . Lo que implica que, aunque típicamente se lo describe como un modelo no supervisado por no requerir etiquetas externas, el aprendizaje es **supervisado**, pues la red debe aprender a reproducir su propia entrada.

En este esquema, el **codificador** proyecta el vector de entrada  $x^{(u)}$  a una representación comprimida  $z^{(u)} \in \mathbb{R}^K$ , donde  $K < D$ , a través de una transformación no lineal:

$$z^{(u)} = \text{codificador}(x^{(u)}), \quad (2.10)$$

mientras que el **decodificador** intenta reconstruir la entrada original generando:

$$\hat{x}^{(u)} = y^{(u)} = \text{decodificador}(z^{(u)}), \quad (2.11)$$

dando lugar al proceso completo de codificación y decodificación.

El entrenamiento del autocodificador se realiza mediante retropropagación, minimizando una función de costo  $\mathcal{L}$  que mide la diferencia entre cada entrada  $x^{(u)}$  y su reconstrucción  $\hat{x}^{(u)}$ . Una forma clásica de esta función está dada por:

$$\mathcal{L} = \frac{1}{ND} \sum_{u=1}^N \sum_{k=1}^D \left( x_k^{(u)} - \hat{x}_k^{(u)} \right)^2, \quad (2.12)$$

equivalente al error cuadrático medio (MSE por sus siglas en inglés) sobre todas las muestras, donde  $N$  es la cantidad de registros y  $D$  es la dimensión de  $x$ . Este error descrito en la Ecuación 2.12 guía el ajuste

de los parámetros del codificador y del decodificador para mejorar la calidad de reconstrucción.

La **no linealidad** presente en el autocodificador surge del uso de funciones de activación aplicadas en cada neurona del modelo. En esencia, cada unidad realiza la operación mostrada en la Ecuación 2.13.

$$a_j^{(u)} = g\left(\sum_{k=1}^D w_{jk} x_k^{(u)}\right), \quad (2.13)$$

donde  $x_k^{(u)}$  representa el valor de la  $k$ -ésima componente del vector de entrada  $x^{(u)}$  que ingresa a la neurona  $j$ ,  $w_{jk}$  es el peso sináptico que regula la contribución de dicha entrada en la activación y  $a_j^{(u)}$  la activación obtenida en dicha neurona. La función  $g(x)$  es una función de activación no lineal por ejemplo,  $\tanh(\beta x)$  o funciones sigmoideas relacionadas que permiten que la red modele relaciones complejas que no podrían representarse mediante transformaciones puramente lineales. Este componente no lineal es esencial para que el autocodificador pueda aprender estructuras internas del conjunto de datos y generar representaciones latentes útiles.

En este trabajo, la función de activación utilizada en todas las capas ocultas es GELU, debido a sus ventajas frente a la comúnmente empleada ReLU. Entre estas ventajas se encuentran: una superficie de pérdida más suave que facilita un entrenamiento más estable, la ausencia del problema de “neuronas muertas” asociado a ReLU y una mayor capacidad para capturar relaciones no lineales gracias a su formulación probabilística basada en la distribución normal.

La función GELU( $x$ ) está definida por:

$$\text{GELU}(x) = 0,5 x \left( 1 + \tanh\left(\sqrt{\frac{2}{\pi}} (x + 0,044715x^3)\right) \right). \quad (2.14)$$

La función ReLU( $x$ ) se define como:

$$\text{ReLU}(x) = \max(0, x). \quad (2.15)$$

Como se muestra en la Figura 2.6, al graficar ambas funciones se aprecia la suavidad de GELU al aproximarse a 0, en contraste con el comportamiento abrupto de ReLU, lo que evita discontinuidades y favorece un entrenamiento más estable en modelos neuronales.

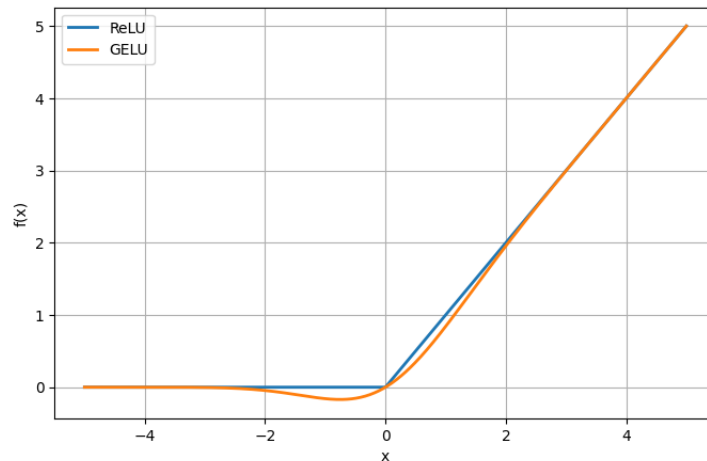


Figura 2.6: Funciones de activación ReLU y GELU.

Si bien el autocodificador estándar es efectivo para reducir la dimensionalidad, asegurando que la capa latente sea de menor dimensión ( $K < D$ ), existen variantes que imponen restricciones adicionales

sobre la representación latente  $\mathbf{z}^{(u)} \in \mathbb{R}^K$  para evitar que el modelo simplemente aprenda la función identidad y para forzarlo a capturar características más robustas y útiles de los datos.

El **Autocodificador Disperso (SAE)** [9] impone una restricción de **dispersión** (sparsity) sobre la capa oculta con  $j, \dots, H$  unidades o neuronas. En este caso, el objetivo no es necesariamente reducir la dimensión ( $K$  puede ser menor o incluso mayor que  $D$ ), sino asegurar que la mayoría de las neuronas en la capa latente permanezcan **inactivas** (con activación cercana a cero) para cada ejemplo  $x^{(u)}$ . Esta dispersión fuerza al modelo a utilizar diferentes subconjuntos de neuronas para representar distintas entradas, lo que previene el sobre ajuste y favorece la extracción de características más discriminativas.

La función de costo total del SAE se denota como  $\mathcal{L}_{\text{sparse}}(W, b)$  y está compuesta por el término de reconstrucción  $\mathcal{L}(W, b)$  más un término de penalización de dispersión:

$$\mathcal{L}_{\text{sparse}}(W, b) = \mathcal{L}(W, b) + \beta \sum_{j=1}^H \text{KL}(\rho \parallel \hat{\rho}_j), \quad (2.16)$$

donde  $W$  denota el conjunto de **matrices de pesos** de todas las capas del autocodificador (tanto del codificador como del decodificador),  $b$  corresponde al conjunto de **vectores de sesgo** o *bias* asociados a dichas capas,  $\beta$  es el hiperparámetro que controla el peso de la penalización y  $H$  son las unidades de la capa oculta. El término de penalización es la suma de la **divergencia Kullback-Leibler** (KL) entre la tasa de activación promedio deseada  $\rho$ , con un valor pequeño fijo (0.05 en el artículo original) y la tasa de activación promedio real ( $\hat{\rho}_j$ ) de cada neurona  $j$  en la capa oculta:

$$\sum_{j=1}^H \text{KL}(\rho \parallel \hat{\rho}_j) = \sum_{j=1}^H \left[ \rho \log \left( \frac{\rho}{\hat{\rho}_j} \right) + (1 - \rho) \log \left( \frac{1 - \rho}{1 - \hat{\rho}_j} \right) \right], \quad (2.17)$$

y  $\hat{\rho}_j$  se calcula como:

$$\hat{\rho}_j = \frac{1}{N} \sum_{u=1}^N a_j^{(u)}, \quad (2.18)$$

esta penalización tiene la propiedad de ser cero si  $\hat{\rho}_j = \rho$ , y aumentar a medida que  $\hat{\rho}_j$  se aleja de  $\rho$ . Al minimizar esta función, el SAE es obligado a tener representaciones latentes dispersas, debido a que la mayoría de las activaciones de las unidades se acercan a 0.

Otra variante interesante es el **Autocodificador Contractivo (CAE)**[10], este se enfoca en hacer que la representación latente  $z^{(u)}$  sea **robusta** frente a pequeñas variaciones o ruido en los datos de entrada  $x^{(u)}$ . El objetivo es que la función de codificación  $f$  sea **contractiva**, es decir, que un pequeño cambio en el vector de entrada produzca un cambio muy pequeño en la representación latente. Esto obliga al modelo a capturar únicamente los **factores de variación más relevantes**, ignorando perturbaciones irrelevantes.

El CAE introduce un término de penalización basado en la sensibilidad del codificador. Esta sensibilidad está dada por la **matriz Jacobiana** del codificador evaluada en cada ejemplo  $x^{(u)}$ :

$$J_f(x^{(u)}) = \frac{\partial f(x^{(u)})}{\partial x^{(u)}} \in \mathbb{R}^{K \times D}. \quad (2.19)$$

La penalización se define como el **cuadrado de la norma de Frobenius** del Jacobiano. Para una matriz  $A \in \mathbb{R}^{m \times n}$ , la norma de Frobenius se define como:

$$\|A\|_F^2 = \sum_{i=1}^m \sum_{j=1}^n A_{ij}^2. \quad (2.20)$$

Con esta definición, la función de costo total del CAE es:

$$\mathcal{L}_{\text{CAE}}(W, b) = \mathcal{L}(W, b) + \lambda \sum_{u=1}^N \left\| J_f(x^{(u)}) \right\|_F^2. \quad (2.21)$$

Donde  $\mathcal{L}(W, b)$  es el error de reconstrucción (por ejemplo, MSE) y  $\lambda > 0$  controla el peso del término contractivo.

Al minimizar  $\mathcal{L}_{\text{CAE}}(W, b)$ , el codificador aprende un mapeo  $x^{(u)} \mapsto z^{(u)}$  que es **suave, estable y resistente al ruido**, haciéndose insensible a variaciones irrelevantes y destacando las características realmente importantes.

A partir de estas arquitecturas, el autocodificador y sus variantes pueden emplearse como un método de **detección de anomalías**. Para ello, se deben tener en cuenta las etiquetas de cada ejemplo del conjunto de datos, definidas en la Ecuación 2.22:

$$y^{(u)} = \begin{cases} 0, & \text{representa un caso benigno (normal)} \\ 1, & \text{representa un caso maligno (anomalía)} \end{cases} \quad (2.22)$$

Con esta definición, el modelo se entrena exclusivamente utilizando el subconjunto de ejemplos que representan el comportamiento “normal”, tal como se especifica en la Ecuación 2.23:

$$\mathbf{X}_{\text{benigno}} = \{ x^{(u)} \in \mathbf{X} \mid y^{(u)} = 0 \}. \quad (2.23)$$

Es decir, masas tumorales benignas. Al aprender a reconstruir correctamente este tipo de patrones, cuando el modelo recibe un nuevo dato  $t$  perteneciente a un conjunto  $\mathbf{T}$  con la misma distribución que  $\mathbf{X}$ , se espera que el error de reconstrucción  $\|\hat{t} - t\|$  sea pequeño.

En contraste, si se presenta un dato atípico  $t^*$ , correspondiente a un diagnóstico maligno, el modelo no logra reconstruirlo con la misma fidelidad, generando así un error significativamente mayor. Este comportamiento permite establecer un criterio simple y efectivo para la detección de anomalías: un dato se clasifica como anómalo si su error de reconstrucción supera un umbral  $\epsilon$  predefinido. El criterio se expresa formalmente en la Ecuación 2.24:

$$\|\hat{t}^* - t^*\| > \epsilon, \quad (2.24)$$

donde  $\hat{t}^*$  es la reconstrucción producida por la red a partir del dato  $t^*$ . Si la desigualdad se cumple, el dato se considera **anómalo**; de lo contrario, se clasifica como **normal**.

El valor del umbral  $\epsilon$  se determinó de manera empírica utilizando el conjunto de validación compuesto únicamente por muestras normales. Para ello, se calcula el error de reconstrucción de cada ejemplo, definido en la Ecuación 2.25.

$$e^{(u)} = \frac{1}{D} \sum_{k=1}^D \left( x^{(u)}_k - \hat{x}^{(u)}_k \right)^2, \quad e = \left( e^{(1)}, \dots, e^{(N_{\text{benigno}})} \right), \quad (2.25)$$

donde  $x^{(u)}$  corresponde a cada muestra del subconjunto  $\mathbf{X}_{\text{benigno}}$  de validación,  $\hat{x}^{(u)}$  a su respectiva reconstrucción generada por el modelo,  $e$  al vector que contiene los errores de reconstrucción de todos los ejemplos normales y  $N_{\text{benigno}}$  a la cantidad de ejemplos del conjunto  $\mathbf{X}_{\text{benigno}}$ . Este vector permite caracterizar el comportamiento esperado de la red frente a datos no anómalos, facilitando la identificación de aquellos casos cuyo error de reconstrucción excede significativamente lo aprendido durante el entrenamiento.

Se definen tres estrategias para calcular el umbral  $\epsilon$ . La primera consiste en calcularlo como la media del error  $e$  más  $q$  veces su desvío estándar. Esta definición se muestra en la Ecuación 2.26.

$$\epsilon = \mu_e + q \cdot \sigma_e, \quad q \in \mathbb{N}. \quad (2.26)$$

La segunda opción utiliza percentiles para establecer el umbral  $\epsilon$ . Los percentiles dividen el conjunto de datos en cien partes iguales y es el valor que debe tomar una variable para estar dentro del  $x\%$  de los valores superiores.

En este caso se utiliza el percentil 95 ( $P_{95\%}$ ), el cual separa al 95 % de los errores de reconstrucción más bajos del 5 % más alto.

$$\epsilon = P_{95\%}(e). \quad (2.27)$$

La tercera y última estrategia empleada para calcular el umbral  $\epsilon$ , se basa en utilizar el **índice de Youden**, un criterio basado en el desempeño del clasificador para cada posible valor de umbral  $\nu$ . Sea  $R_\nu$  el *recall* y  $S_\nu$  la *specificity* obtenidas al evaluar el modelo con dicho umbral. El índice de Youden maximiza simultáneamente ambas métricas. Se define en la Ecuación 2.28.

$$\epsilon = \arg \max_{\nu} \{R_\nu + S_\nu - 1\}. \quad (2.28)$$

### 2.2.2. Métricas de evaluación del modelo

Para evaluar rigurosamente el desempeño del autocodificador en la tarea de detección de anomalías, se analizan diversas métricas derivadas del error de reconstrucción  $\|\hat{t} - t\|$ . Dado que este valor actúa como cuantificador directo de qué tan bien el modelo reproduce cada instancia, es posible utilizarlo como criterio de decisión para distinguir entre casos normales y anómalos mediante un umbral  $\epsilon$ .

A partir de esta regla se construye la matriz de confusión, desde la cual se obtienen métricas clásicas de evaluación como la exactitud, la sensibilidad y la especificidad. Asimismo, se emplean herramientas adicionales basadas en la variación del umbral, como la curva ROC y el área bajo dicha curva (AUC) que permiten caracterizar el rendimiento global del modelo sin depender de un valor fijo de  $\epsilon$ , proporcionando una visión más completa de su capacidad discriminativa en términos del error de reconstrucción.

Para obtener un criterio de decisión utilizamos el umbral  $\epsilon$  definido en las Ecuaciones 2.26, 2.27, 2.28, a partir del cual, cada instancia se clasifica como **normal** o **anómala** según si su error de reconstrucción  $\|\hat{t} - t\|$  supera o no el umbral  $\epsilon$ , tal como se muestra en la Ecuación 2.29.

$$\hat{y} = \begin{cases} 0, & \text{si } \|\hat{t} - t\| \leq \epsilon \\ 1, & \text{si } \|\hat{t} - t\| > \epsilon \end{cases}. \quad (2.29)$$

A partir de este criterio, se distinguen los siguientes cuatro casos fundamentales:

- **Verdaderos Positivos (TP)**: Casos *anómalos* (malignos) cuyo error de reconstrucción satisface  $\|\hat{t} - t\| > \epsilon$ .
- **Falsos Negativos (FN)**: Casos *anómalos* (malignos) cuyo error cumple  $\|\hat{t} - t\| \leq \epsilon$ , es decir, el modelo no detectó la anomalía.
- **Verdaderos Negativos (TN)**: Casos *normales* (benignos) cuyo error satisface  $\|\hat{t} - t\| \leq \epsilon$ .
- **Falsos Positivos (FP)**: Casos *normales* (benignos) cuyo error satisface  $\|\hat{t} - t\| > \epsilon$ , siendo incorrectamente clasificados como anómalos.

A partir de estas cuatro categorías se construye la matriz de confusión mostrada en la Tabla 2.5, donde la diagonal verde representa las predicciones correctas y las celdas rojas los errores de clasificación.

Tabla 2.5: Matriz de confusión. La diagonal verde representa predicciones correctas, mientras que las celdas rojas corresponden a errores de clasificación.

|      |         | Predicción              |                         |
|------|---------|-------------------------|-------------------------|
|      |         | Anómalo                 | Normal                  |
| Real | Anómalo | Verdadero Positivo (TP) | Falso Negativo (FN)     |
|      | Normal  | Falso Positivo (FP)     | Verdadero Negativo (TN) |

Una vez definidas las categorías de la matriz de confusión, es posible cuantificar el rendimiento del modelo mediante distintas métricas, cada una evaluando un aspecto particular de la calidad de las predicciones. Todas ellas se derivan directamente de los valores TP, FP, TN y FN.

La **exactitud** (*Accuracy*) cuantifica la proporción total de predicciones correctas realizadas por el modelo según la Ecuación 2.30.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}}. \quad (2.30)$$

La precisión (*Precision*) indica qué proporción de las predicciones positivas corresponde realmente a casos positivos, tal como se define en la Ecuación 2.31.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}. \quad (2.31)$$

La sensibilidad (*Recall*) mide la capacidad del modelo para identificar correctamente los casos positivos presentes en el conjunto de datos, de acuerdo con la Ecuación 2.32.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}. \quad (2.32)$$

La especificidad (*Specificity*) evalúa la proporción de casos negativos correctamente clasificados, definida en la Ecuación 2.33.

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}. \quad (2.33)$$

La exactitud balanceada (*Balanced Accuracy*), mostrada en la Ecuación 2.34, combina sensibilidad y especificidad para obtener una medida equilibrada del rendimiento.

$$\text{Balanced Accuracy} = \frac{\text{Recall} + \text{Specificity}}{2}. \quad (2.34)$$

El *F1-Score* combina las medidas de *Precision* y *Recall* para ofrecer una métrica más general de la calidad del modelo, tal como se expresa en la Ecuación 2.35.

$$\text{F1}_{\text{score}} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \text{ TP}}{2 \text{ TP} + \text{FP} + \text{FN}}. \quad (2.35)$$



En el caso particular de este trabajo, resulta crítico priorizar la métrica de *Recall*. Dado que un falso negativo implica no identificar un caso potencialmente maligno. Maximizar esta métrica es esencial para garantizar que el modelo no pase por alto ninguna anomalía.

Si bien *Precision* también aporta información valiosa al reducir el número de falsos positivos, la severidad clínica de un falso negativo vuelve al *Recall* la métrica de mayor importancia en este contexto. No obstante, para obtener una visión más equilibrada del rendimiento global del sistema, también se considera el  $F1_{score}$ , que combina *Recall* y *Precision* en una única medida. De esta manera, aunque el objetivo principal es maximizar el *Recall*, un  $F1_{score}$  elevado asegura que dicho incremento no se logra a costa de elevar los falsos positivos, proporcionando así una evaluación más completa de la calidad del modelo.

Las métricas presentadas hasta ahora (*Accuracy*, *Precision*, *Recall*, *Specificity*, etc.) son dependientes del umbral de decisión  $\epsilon$  elegido. Para obtener una evaluación del rendimiento que sea independiente de la elección de un umbral específico y que caracterice la capacidad discriminativa del modelo en todo el rango de posibles valores del error de reconstrucción, se utiliza la **Curva ROC** y el área bajo esta curva (**AUC**).

La **Curva ROC** es una representación gráfica que resume el comportamiento del clasificador al comparar la proporción de verdaderos positivos contra la proporción de falsos positivos para distintos valores de umbral  $\epsilon$ . Para construirla, se utiliza en el eje vertical *y* la **Tasa de Verdaderos Positivos** (o por sus siglas en inglés TPR de *True Positive Rate*), una medida que cuantifica qué tan eficaz es el modelo para detectar correctamente las anomalías presentes en los datos. Esta tasa se define en la Ecuación 2.36.

$$TPR = \frac{TP}{TP + FN} = \frac{TP}{P}. \quad (2.36)$$

De forma análoga, el eje horizontal *x* de la curva utiliza la **Tasa de Falsos Positivos** (o por sus siglas en inglés FPR de *False Positive Rate*), que indica cuántos casos normales son clasificados erróneamente como anómalos. Se describe en la Ecuación 2.37 y esta cantidad es complementaria a la especificidad definida anteriormente en la Ecuación 2.33.

$$FPR = \frac{FP}{FP + TN} = \frac{FP}{N} = 1 - \text{Specificity}. \quad (2.37)$$

La curva se genera variando el umbral de decisión  $\epsilon$  a través de todos los valores posibles del error de reconstrucción. Cada valor de  $\epsilon$  resulta en un punto diferente (FPR, TPR) en el espacio ROC. Un clasificador ideal se encuentra en el punto (0, 1) del espacio ROC, lo que corresponde a  $TPR = 1$  (todos los anómalos detectados) y  $FPR = 0$  (ningún normal clasificado erróneamente).

El **Área Bajo la Curva ROC (AUC)** es una métrica escalar que resume el rendimiento global del modelo en una única medida. Se define como el área total delimitada por la curva ROC y los ejes.

El AUC proporciona una medida robusta e independiente del umbral de la capacidad de clasificación del modelo, lo que lo convierte en una métrica fundamental para evaluar el desempeño general. Especialmente en tareas de detección de anomalías, donde la distribución de clases suele ser altamente desbalanceada. Un clasificador es razonable si  $0,5 < AUC \leq 1$ .



## Capítulo 3

# Resultados

En este capítulo se presentan los resultados obtenidos durante el entrenamiento de los autocodificadores propuestos. Para ello, se muestran las configuraciones utilizadas, los histogramas de errores de reconstrucción y finalmente las métricas de desempeño para los distintos modelos y sus variantes entrenadas con los conjuntos de diferentes proporciones, que resumen el rendimiento de cada modelo bajo el criterio de decisión.

Para cada una de las arquitecturas consideradas: autocodificador clásico (AE), autocodificador disperso (SAE) y autocodificador contractivo (CAE) se entrenan tres variantes (A, B y C), que comparten exactamente la misma configuración e hiperparámetros y difieren únicamente en el conjunto de entrenamiento utilizado.

Las configuraciones finales empleadas para cada arquitectura surgen de un proceso de *grid search*, en el cual se exploraron distintas combinaciones de cantidad de capas, de neuronas y tasas de aprendizaje (*lr* por sus siglas en inglés de *Learning Rate*). La selección de los hiperparámetros óptimos se basó en el desempeño sobre el conjunto de **validación**, priorizando las métricas de *Recall*, *F1<sub>score</sub>* y AUC. Posteriormente, los modelos seleccionados fueron entrenados con los hiperparámetros con mejores métricas y fueron evaluados sobre el conjunto de prueba, cuyos resultados son los expuestos en este capítulo. La Tabla 3.1 resume los hiperparámetros utilizados para cada una de las arquitecturas finales:

Tabla 3.1: Hiperparámetros de entrenamiento para cada modelo.

| Modelo | Dimensiones de capas  | <i>lr</i> | <i>Batch size</i> | Épocas |
|--------|-----------------------|-----------|-------------------|--------|
| AE     | [30, 32, 16, 8, 4, 2] | 0.001     | 16                | 200    |
| SAE    | [30, 64]              | 0.001     | 16                | 200    |
| CAE    | [30, 32, 16]          | 0.001     | 16                | 200    |

En todos los modelos se utiliza **MSE** como función de pérdida, **Adam** como optimizador y una **paciencia** para el *Early Stop* de 20 épocas.

Para cada uno de los modelos entrenados se calculan los diversos umbrales  $\epsilon$ , que permiten distinguir entre instancias normales y anómalas a partir del error de reconstrucción. Los umbrales se obtienen utilizando únicamente el conjunto de validación, siguiendo los criterios definidos en las Ecuaciones 2.26, 2.27 y 2.28. Los umbrales empleados por cada modelo se muestran en la Tabla 3.2.

Tabla 3.2: Valores de umbral  $\epsilon$  obtenidos con diferentes métodos y modelos.

| Modelo  | Media | Media + 2 · desvío | Media + 3 · desvío | Percentil 95 | Youden |
|---------|-------|--------------------|--------------------|--------------|--------|
| AE (A)  | 0.227 | 0.665              | 0.884              | 0.657        | 0.446  |
| AE (B)  | 0.304 | 0.890              | 1.183              | 0.953        | 0.313  |
| AE (C)  | 0.335 | 0.983              | 1.308              | 1.024        | 0.334  |
| SAE (A) | 0.132 | 0.356              | 0.467              | 0.334        | 0.762  |
| SAE (B) | 0.161 | 0.453              | 0.599              | 0.478        | 0.181  |
| SAE (C) | 0.124 | 0.348              | 0.460              | 0.295        | 0.242  |
| CAE (A) | 0.019 | 0.095              | 0.133              | 0.052        | 0.037  |
| CAE (B) | 0.023 | 0.092              | 0.126              | 0.065        | 0.042  |
| CAE (C) | 0.024 | 0.089              | 0.121              | 0.100        | 0.021  |

Una vez definido el umbral para cada modelo, se analiza la distribución del error de reconstrucción sobre el conjunto de prueba para los **autocodificadores clásicos**. Los histogramas mostrados en las Figuras 3.1, 3.2 y 3.3 ilustran la distribución de dichos errores, distinguiendo los registros normales (en azul) de los anómalos (en naranja), así como la ubicación de los umbrales  $\epsilon$  en cada caso. Esta visualización permite observar la capacidad discriminativa de los distintos umbrales.

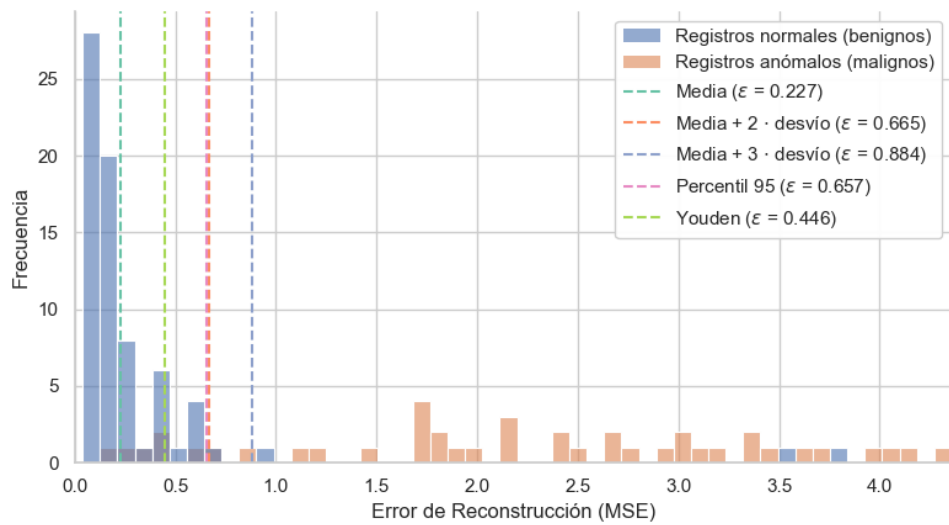


Figura 3.1: Distribución del error de reconstrucción para el autocodificador A.

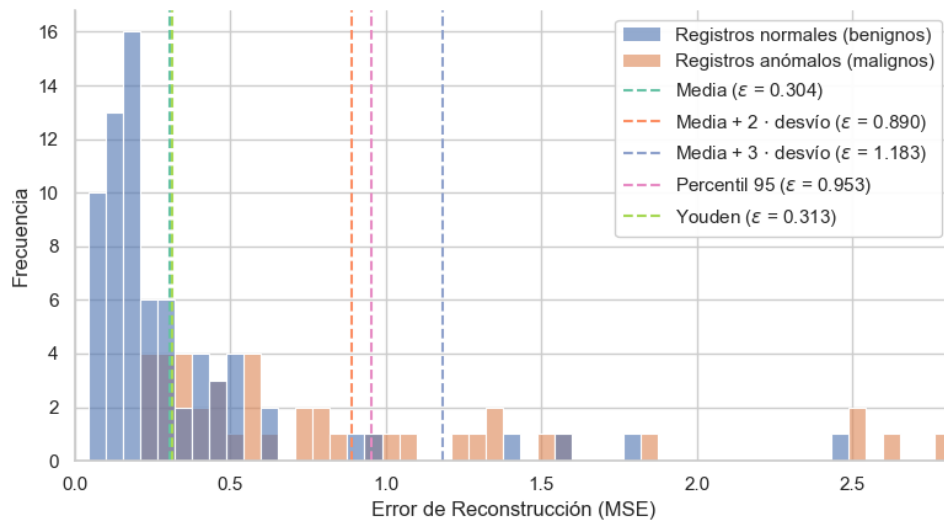


Figura 3.2: Distribución del error de reconstrucción para el autocodificador B.

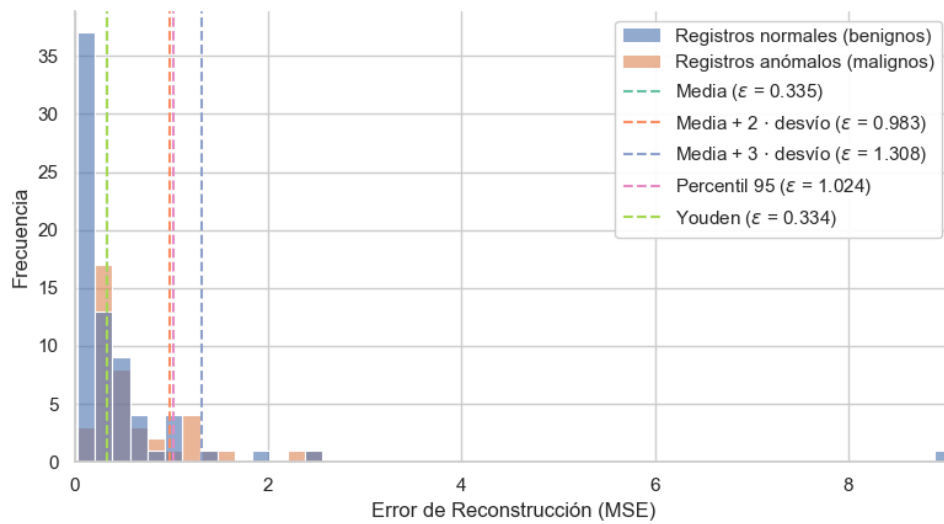


Figura 3.3: Distribución del error de reconstrucción para el autocodificador C.

Por su capacidad discriminadora en todos los conjuntos se utiliza el  $\epsilon$  calculado con el índice de Youden. A partir del mismo, se calculan las métricas para cada modelo, se muestran en la Tabla 3.3.

Tabla 3.3: Métricas de rendimiento para cada modelo entrenado con los conjuntos A, B y C.

| Modelo  | <i>Accuracy</i> | <i>Precision</i> | <i>Recall</i> | <i>Specificity</i> | <i>Balanced Accuracy</i> | <i>F1<sub>score</sub></i> |
|---------|-----------------|------------------|---------------|--------------------|--------------------------|---------------------------|
| AE (A)  | 0.860           | 0.771            | 0.881         | 0.847              | 0.864                    | 0.822                     |
| AE (B)  | 0.746           | 0.618            | 0.810         | 0.708              | 0.759                    | 0.701                     |
| AE (C)  | 0.632           | 0.500            | 0.619         | 0.639              | 0.629                    | 0.553                     |
| SAE (A) | 0.939           | 0.973            | 0.857         | 0.986              | 0.922                    | 0.911                     |
| SAE (B) | 0.754           | 0.652            | 0.714         | 0.778              | 0.746                    | 0.682                     |
| SAE (C) | 0.640           | 0.538            | 0.167         | 0.917              | 0.542                    | 0.255                     |
| CAE (A) | 0.930           | 0.886            | 0.929         | 0.931              | 0.930                    | 0.907                     |
| CAE (B) | 0.798           | 0.757            | 0.667         | 0.875              | 0.771                    | 0.709                     |
| CAE (C) | 0.693           | 0.566            | 0.714         | 0.681              | 0.697                    | 0.632                     |

Donde se observa que el mayor *Recall* lo posee el modelo **CAE (A)** y el modelo **SAE (A)** cuenta con el mayor *F1<sub>score</sub>*, *Precision* y *Specificity*. También se visualiza el decrecimiento de las métricas al aumentar las anomalías en el entrenamiento de los modelos, mayormente en el modelo SAE.

Finalmente, con el objetivo de evaluar la capacidad discriminativa de cada arquitectura frente a instancias anómalas independientemente del umbral  $\epsilon$  seleccionado, se calculan las curvas ROC y sus correspondientes valores de AUC para todos los modelos entrenados. Para facilitar la comparación, los modelos se agrupan según el conjunto de entrenamiento utilizado (A, B y C), lo que permite analizar cómo varía el rendimiento ante distintos niveles de contaminación en el entrenamiento. Los resultados se presentan en las Figuras 3.4, 3.5 y 3.6. Donde se observa que el *contractive autoencoder* tiene una mayor robustez que el resto de los modelos al entrenarse con anomalías.

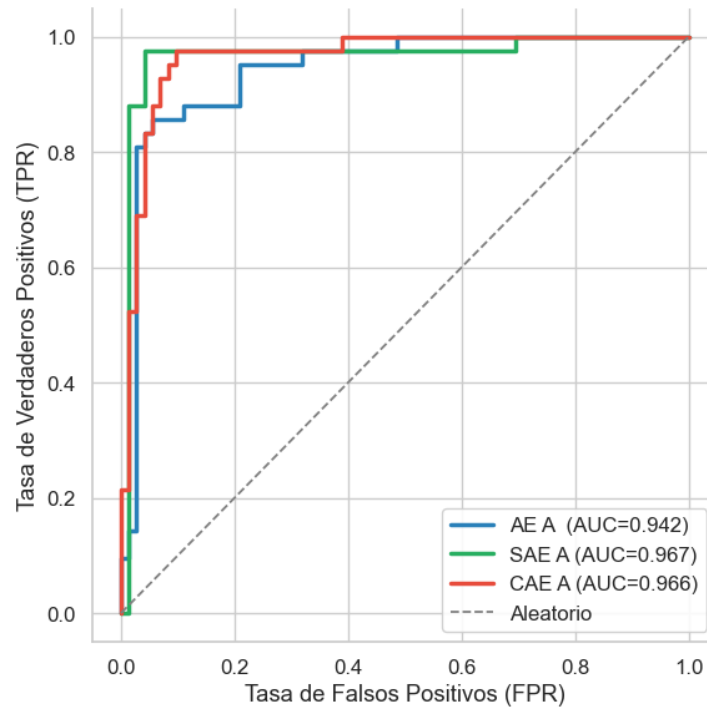


Figura 3.4: Curvas ROC y AUC obtenidos para los modelos AE, SAE y CAE entrenados con el Conjunto A (0 % de anomalías).

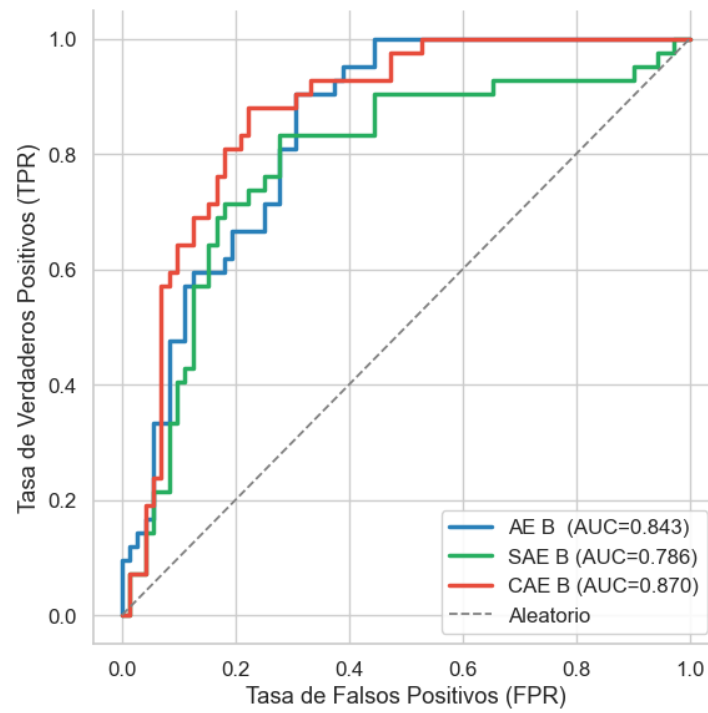


Figura 3.5: Curvas ROC y AUC obtenidos para los modelos AE, SAE y CAE entrenados con el Conjunto B (10 % de anomalías).

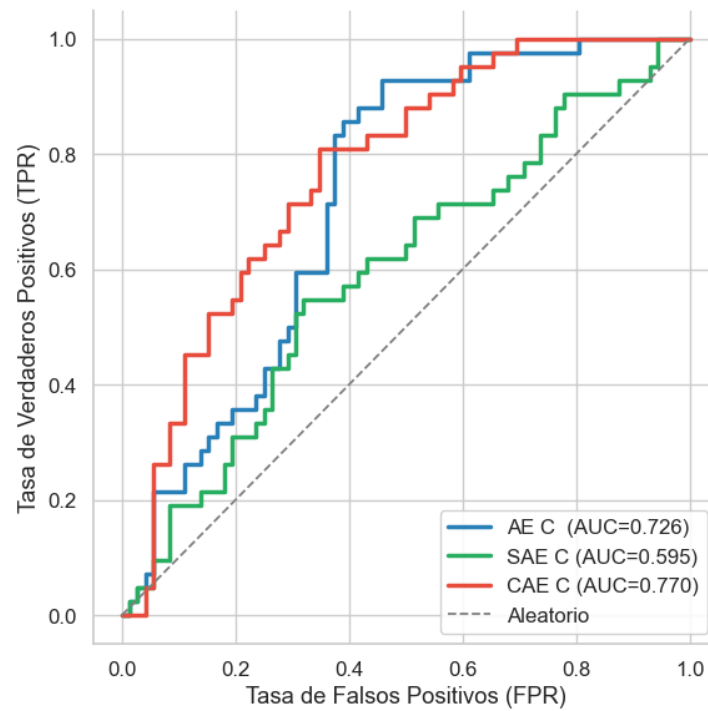


Figura 3.6: Curvas ROC y AUC obtenidos para los modelos AE, SAE y CAE entrenados con el Conjunto C (25 % de anomalías).





## Capítulo 4

# Conclusiones

El presente trabajo exploró el uso de autocodificadores (*Autoencoders*) para la detección de anomalías, aplicados sobre un conjunto de datos clínicos tras un proceso de análisis exploratorio y preprocesamiento. Se estudiaron tres variantes del modelo: clásico, *sparse* y *contractive* con el objetivo de evaluar cómo las diferencias en las arquitecturas afectan la capacidad del modelo para identificar patrones propios de los registros normales y distinguirlos de las anomalías.

Los resultados experimentales muestran una buena capacidad discriminativa de los modelos, destacando por sus métricas generales al SAE y por su *Recall* junto a su robustez ante la presencia de anomalías al CAE. Por otra parte, el autoencoder clásico presenta un resultado aceptable en las métricas, teniendo mayor robustez al ruido que el SAE pero sin destacar en ninguna métrica.

También, se destaca la importancia de la pureza de los datos a la hora de entrenar detectores de anomalías destinados a aplicaciones reales, debido a las implicaciones que tienen en los modelos al estar presentes.

En relación a la capacidad discriminativa de los modelos, se enfatiza la importancia de la selección del umbral  $\epsilon$ , donde su elección puede afectar totalmente las métricas obtenidas para cada modelo, siendo un mejor indicativo de su capacidad para distinguir anomalías el valor de AUC.

Dentro de las opciones para calcular el umbral  $\epsilon$ , se destaca el índice de Youden, debido a su capacidad para maximizar las métricas de *Recall* y *Specificity*, aunque cada umbral puede priorizar distintas métricas.

Aunque los resultados alcanzados son satisfactorios, quedan abiertas múltiples líneas de investigación, como la aplicación de nuevos enfoques para la detección de anomalías o el estudio en distintos tipos de datos (como imágenes o series de tiempo), siguiendo las bases planteadas en este trabajo.



# Bibliografía

- [1] Dor Bank, Noam Koenigstein, and Raja Giryes. Autoencoders. *arXiv preprint arXiv:2003.05991*, 2020.
- [2] Roel Bouman and Tom Heskes. Autoencoders for anomaly detection are unreliable. *arXiv preprint arXiv:2501.13864*, 2025.
- [3] H Bourlard and Y Kamp. Auto-association by multilayer perceptrons and singular value decomposition. *Biol. Cybern*, 59:291–294, 1988.
- [4] Aristidis Diamantis, Emmanouil Magiorkinis, and Helen Koutselini. Fine-needle aspiration (fna) biopsy: historical aspects. *Folia histochemica et cytobiologica*, 47(2):191–197, 2009.
- [5] KS GGCS. Learning representations by back-propagating errors. *NATURE*, 323(9), 1986.
- [6] UCI Machine Learning and Ovsen. Breast cancer wisconsin (diagnostic) data set. <https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data/data>. Kaggle Dataset. Accedido: 2025-11-27.
- [7] Ali Nawaz, Shehroz S. Khan, and Amir Ahmad. Ensemble of autoencoders for anomaly detection in biomedical data: A narrative review. *IEEE Access*, 12:17273–17289, 2024.
- [8] Asif Ahmed Neloy and Maxime Turgeon. A comprehensive study of auto-encoders for anomaly detection: Efficiency and trade-offs. *Machine Learning with Applications*, 17:100572, 2024.
- [9] Andrew Ng et al. Sparse autoencoder. *CS294A Lecture notes*, 72(2011):1–19, 2011.
- [10] Salah Rifai, Pascal Vincent, Xavier Muller, Xavier Glorot, and Yoshua Bengio. Contracting autoencoders. In *Proceedings of the International Conference on Machine Learning (ICML), Bellevue, WA, USA*, page 95, 2011.
- [11] William Nick Street, William H. Wolberg, and Olvi L. Mangasarian. Nuclear feature extraction for breast tumor diagnosis. In *Electronic imaging*, 1993.
- [12] Tomasz Walczyna, Damian Jankowski, and Zbigniew Piotrowski. Enhancing anomaly detection through latent space manipulation in autoencoders: A comparative analysis. *Applied Sciences*, 15(1):286, 2024.
- [13] William Wolberg, Olvi Mangasarian, Nick Street, and W. Street. Breast cancer wisconsin (diagnostic). UCI Machine Learning Repository, 1993. DOI: <https://doi.org/10.24432/C5DW2B>.